NetResQ

An Adaptive Self-Healing Network Using Quality-Aware Routing

OSI LAYER 3 (NETWORK LAYER) SOFTWARE-DEFINED NETWORKING (SDN) FASTAPI + NETWORKX + REACT 19

Project Name: NetResQ (Network Rescue)

Academic Domain: Computer Networks & SDN

Primary Routing Core: NetworkX (Graph Theory & Dijkstra)

Control Plane: FastAPI REST Backend + Ryu OpenFlow 1.3

Presentation Dashboard: React 19 + TypeScript + Vite

Target Emulation: Standalone Simulation / Mininet + OVS

1. Executive Summary & Problem Statement

In conventional computer networks, interior gateway routing protocols such as **OSPF (Open Shortest Path First)** and **RIP (Routing Information Protocol)** rely on static link costs or hop counts to calculate shortest paths using Dijkstra's algorithm.

The Core Problem (The "Blind Shortest Path" Flaw):
In real-world networks, intermediate links frequently suffer from transient congestion, bufferbloat, physical line noise, or packet drops without suffering a complete physical cable cut. Because conventional shortest-path algorithms only evaluate static configured costs, they continue sending critical traffic over a degraded link as long as its physical port stays UP. This causes severe application degradation, latency spikes, and TCP retransmissions.

> **Example Flaw:** If a direct 1-hop link between Switch 1 and Switch 3 experiences **65ms latency** and **20% packet drops**, standard Dijkstra routing continues to send packets through it simply because it is 1 hop (cost = 1.0), completely ignoring a healthy 2-hop alternate path (via Switch 2) that has only **11ms latency and 0% packet loss**!

2. What NetResQ Does (The Solution)

NetResQ (Network Rescue) solves this challenge by implementing an intelligent, adaptive network control engine that provides:

- **Continuous Link Quality Telemetry:** Tracks four fundamental physical metrics in real time: administrative base cost, latency (ms), packet loss percentage (%), and link utilization (%).
- **Multi-Metric Normalization:** Dimensionless transformation that allows latency, loss percentage, and utilization to be combined mathematically without dimensional skewing.
- **Dynamic Quality-Aware Path Selection:** Employs a composite cost function where links with high loss or latency are penalized, steering traffic over healthier alternative routes.
- **Automated Self-Healing Failover:** Instantly detects link failures, excludes failed edges from the network graph, and recalculates alternate routes in sub-millisecond time.
- **Interactive Operations Dashboard:** A dark-mode operations dashboard built with React 19, allowing operators to visually inspect network topology, manually inject link failures, test congestion, and audit failover events.

3. The Quality-Aware Routing Algorithm & Mathematics

3.1 Metric Normalization

To mathematically combine milliseconds, percentages, and arbitrary administrative costs, each metric is normalized into a dimensionless scale:

```
normalized_latency = latency_ms / latency_reference_ms (default reference = 50.0 ms)
normalized_loss = packet_loss_percent / 100.0 (range: 0.0 to 1.0) normalized_utilization =
utilization_percent / 100.0 (range: 0.0 to 1.0)
```

3.2 Composite Edge-Cost Formula

Each usable link in the network graph is assigned a composite cost:

```
link_cost = (w_cost × C_admin) + (w_latency × norm_latency) + (w_loss × norm_loss) + (w_util ×
norm_util)
```

Default Weight Configuration:

- **w_cost = 1.0:** Baseline administrative hop weight.
- **w_latency = 1.0:** Penalty for propagation and queuing delay.
- **w_loss = 2.0:** High penalty for packet drops to prevent TCP collapse.
- **w_util = 1.5:** Penalty for link saturation to prevent bottleneck formation.

3.3 Mathematical Proof of Dijkstra Correctness

> **Guarantee of Optimal Solution:** Dijkstra's shortest path algorithm is mathematically valid for graphs with non-negative edge weights ($w \geq 0$). Because all our weights $w \geq 0$, and link metrics are physically non-negative, NetResQ enforces a minimum cost floor of MIN_EDGE_COST = 0.001. As a result, no negative edges or cycles can ever exist, and Dijkstra's algorithm is **mathematically guaranteed** to find the global lowest-cost path.

3.4 Worked Numerical Example

Consider routing traffic from **Host 1 (H1)** to **Host 4 (H4)**:

Path Option	Hops	Intermediate Links	Latency	Packet Loss	Calculated Cost	Algorithm Selection
Path A (Direct)	3 hops	H1 → S1 → S3 → H4	67.0 ms	**18.0 %**	**6.12**	Chosen by **Baseline Dijkstra** (fewer hops)
Path B (Detour)	4 hops	H1 → S1 → S2 → S3 → H4	**13.0 ms**	**0.0 %**	**4.79**	Chosen by **Quality-Aware Routing** (superior quality)

Result: Quality-Aware Routing chooses Path B because **4.79 < 6.12**. By taking 1 extra hop through S2, it reduces latency by **54ms** and eliminates **18% packet loss**!

4. Overall Architecture & Implementation

NetResQ is architected into three strictly decoupled tiers:

Tier	Primary Tech	Key Modules & Functionality
Frontend	React 19, TypeScript, Vite, Vanilla CSS	**TopologyView.tsx:** Interactive SVG canvas of switches & hosts. **RoutingComparison.tsx:** Side-by-side algorithm comparison. **MetricCards.tsx:** Real-time telemetry gauges and health scores. **EventLogView.tsx:** Chronological failover audit log. **SettingsView.tsx:** Real-time weight sliders.
Backend API	FastAPI, Uvicorn, Pydantic	**app/main.py:** ASGI server entrypoint with restricted local CORS. **app/api/routes.py:** REST endpoints for topology, compute, fail, restore, reset. **app/models.py:** Pydantic schemas validating all inputs and outputs.
Routing Core & Adapters	Python 3.12, NetworkX	**quality_aware.py:** Composite cost formula & Dijkstra path engine. **shortest_path.py:** Baseline hop-count Dijkstra routing. **simulation_adapter.py:** In-memory virtual topology (standalone mode). **sdn_controller_adapter.py:** Interface to Ryu OpenFlow 1.3 controller.

5. How to Demonstrate NetResQ in 5 Steps

1. **Healthy Baseline:** Select route `H1 → H4`. Observe the green links and the active cyan path taking the direct route (`H1 → S1 → S3 → H4`).
2. **Simulate a Link Failure:** Click link `L_S1_S3` on the diagram and click **"Fail Link"**. The link turns red-dashed. NetResQ instantly self-heals and reroutes traffic through `S2` in **0.48 ms**.
3. **Restore Link:** Click **"Restore Link"**. The link turns green, and the path returns to normal.
4. **Inject Congestion:** Click the **"Demo Congestion"** button in the header. Link `L_S1_S3` is injected with 65ms delay and 18% loss.
5. **Inspect Comparison:** Open the **"Routing Analysis & Proof"** tab. Show the evaluator that Baseline Dijkstra is trapped using the degraded 3-hop link, while NetResQ automatically selects the 4-hop route via S2 to protect user quality.

6. Key Viva & Exam Questions (And How to Answer Them)

Q1: At which layer of the OSI model does NetResQ operate?

> NetResQ operates primarily at **OSI Layer 3 (Network Layer)**, making end-to-end routing decisions across switches and subnets.

Q2: What is the separation of planes in SDN, and how is it used here?

> SDN separates the **Control Plane** (decision-making logic) from the **Data Plane** (packet forwarding hardware). NetResQ's FastAPI/NetworkX core acts as the centralized control plane, while OpenFlow switches act as the data plane.

Q3: Why doesn't NetResQ use Bellman-Ford or Floyd-Warshall?

> Bellman-Ford is slower ($O(V \times E)$) and primarily needed for negative edge weights. Because our composite cost function guarantees strictly positive costs (≥ 0.001), Dijkstra's algorithm ($O(E + V \log V)$) is significantly faster and globally optimal.

Q4: What is the difference between software rerouting time and physical recovery time?

> NetResQ's displayed recovery time (~0.5ms to 1.1ms) is the CPU software execution time to prune the graph and run Dijkstra. Physical recovery in hardware additionally incurs optical/copper link loss detection and OpenFlow rule installation (typically 20ms to 100ms).

Q5: How does the system handle a total network partition?

> If all links leading to a destination fail, `networkx.has_path()` returns `False`. The backend cleanly returns status `"no_path"`, logs a high-severity error event, and displays a user-friendly partition notice on the dashboard without crashing.
